BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Perfomance
Environment
Actuators
Sensors

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.agent_pos
self.walls
self.food_positions
self.opponents

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

This environment is classified as a Multi-Agent environment.

Justification:

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

percept sequence is the historical log of every single percept the agent has received from its initialization up to the current moment

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors (S)

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Partially Observable,
because the agent does not receive the complete state of the environment. It only knows its own position, opponent positions, and whether there is food at its current location (smells food). It cannot directly observe the locations of all remaining food

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

Performance Measure (P)

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

processing state of the agent.
It is structurally important because the performance measure should evaluate the outcome of the agent's actions in the environment, not its internal computation. In this game, hitting a wall changes the agent's interaction with the environment and

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment remains Partially Observable because the agent's sensors (get_percept()) do not provide complete information about the environment state. The agent cannot observe hidden toxic trap locations (and also cannot directly observe all food positions), so it must make decisions with incomplete knowledge

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

he smells_toxin sensor helps the agent maximize expected utility by providing information about potential negative outcomes. With this additional percept, the agent can make more rational decisions by avoiding toxic traps and increasing its chances of achieving a higher score.

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Vacuum World exploit demonstrated that an agent will optimize whatever metric it is given, even if that metric does not represent the true goal. A faulty performance measure can cause the agent to exploit loopholes instead of performing the intended task. This is why our grid environment applies a strong

Finalize and Lock Lab 01 Analytical Evaluation